IMSE 785

Big Data Analytics

Pyspark Wordcount

Shing Chang

IMSE, K-State

Learning Objectives

- Word count example has been used to demonstrate the concept of Map Reduce in the programming style: (key, val) in Hadoop ecosystem
- As we start to use Spark, the same concept remains
- Pyspark codes will be used to demonstrate how (key, val) is executed in Spark
- This module will introduce some key coding ingredients for pyspark programming, i.e., map, lamda function, user-defined function, and spark session.
- You are expected to read pyspark codes

The codes: wordcount.py

- Ref. Bengfort Github https://github.com/bbengfort/hadoop-fundamentals/tree/master/spark
- This code can also be found on Canvas>Files>Python>wordcount
- The key code is just one long statement:

```
19      counts = sotu.flatMap(lambda line: line.split(" ")) \
20                   .map(lambda word: (word, 1)) \
21                   .reduceByKey(lambda a, b: a + b)
22
```

```python
from pyspark import SparkConf, SparkContext
import sys


if __name__ == "__main__":
    if len(sys.argv) != 3:
        print "Incorrect number of arguments, correct usage: wordcount.py [inputfile] [outputfile]"
        sys.exit(-1)

    # set input and dictionary from args
    input = sys.argv[1]
    output = sys.argv[2]

    conf = SparkConf().setMaster("local").setAppName("Word Count")
    sc = SparkContext(conf=conf)

    sotu = sc.textFile(input)

    counts = sotu.flatMap(lambda line: line.split(" ")) \
                 .map(lambda word: (word, 1)) \
                 .reduceByKey(lambda a, b: a + b)

    counts.coalesce(1).saveAsTextFile(output)

    sc.stop()
    print "Done!"
```

The codes: revised wordcount2.py with “add”

- At line 3, this new statement would import the “add” operator

```
1  from pyspark import SparkConf, SparkContext
2  import sys
3  from operator import add
```

- This is the revised counts statement

```
20     counts = sotu.flatMap(lambda line: line.split(" ")) \
21                  .map(lambda word: (word, 1)) \
22                  .reduceByKey(add)
```

- Note that the lambda function is replaced by the “add” operator on line 22

Submit a pyspark job

- Put all files e.g. the program and data into hdfs
- If you use the directory to host those files, change the current directory to that directory
- At the VM prompt $
- spark-submit wordcount.py test.txt out
- Note that test.txt is the input file and out is output directory
- To see the result, at the VM prompt $
- hdfs dfs -cat out/part-00000

A tutorial using line-by-line statements

- Ref. wordcount3.ipython
- #read in a text file

text=sc.textFile("test.txt")

- #ReducedByKey is used to compute the wordcount; add was imported in line 7

counts=wc.reduceByKey(add)

counts.saveAsTextFile("outwc")

- #Exit pyspark by entering quit()
- #at VM prompt $ hdfs dfs -ls outwchdfs
- #at VM prompt $ hdfs dfs -cat outwc/part* > part-all
- #note that part-all is in the local machine not in HDFS
- #at VM prompt $ cat part-all

```
2  text=sc.textFile("test.txt")
3
4  #now text is a rdd
5  print text
6
7  from operator import add
8
9  def tokenlize(text):
10     return text.split()
11
12 #flatMap applies to each word/token
13 words=text.flatMap(tokenlize)
14 words.take(3)
15
16 #map applies to each row of words/tokens
17 #for comparison purpose only
18 words2=text.map(tokenlize)
19 words2.take(3)
20
21 #Use map to create (key, val) pairs
22 wc=words.map(lambda x: (x,1))
23
24 #toDebugString is used to reveal the RDD pipeline (ie. PipelinedRDD)
25 print wc.toDebugString()
26
27 #ReducedByKey is used to compute the wordcount; add was imported in line 7
28 counts=wc.reduceByKey(add)
29 counts.saveAsTextFile("outwc")
30
31 #Exit pyspark by entering quit()
32 #at VM prompt $ hdfs dfs -ls outwchdfs
33 #at VM prompt $ hdfs dfs -cat outwc/part* > part-all
34 #note that part-all is in the local machine not in HDFS
35 #at VM prompt $ cat part-all
```

SparkContext and SparkConf (Spark 1.x)

- SparkContext is the entry point for spark environment
- SparkConf contains the cluster configs and parameters to create a Spark Context object
- A Spark session is a combination of all different contexts including SQLContext, HiveContext, Streaming Applications,… etc. So in Spark v2, we can use a spark session instead of SparkContext.
- Internally, Spark session creates a new SparkContext for all the operations.
- We don’t have to create a spark session object when using spark-shell because it is already created with the variable spark. Try, scalar> spark and scalar> spark conf

SparkContext and SparkConf (Spark 1.x)

- //set up the spark configuration and create contexts val sparkConf = new SparkConf().setAppName("SparkSessionZipsExample").setMaster("local") // your handle to SparkContext to access other context like SQLContext val sc = new
- SparkContext(sparkConf).set("spark.some.config.option", "some-value") val sqlContext = new org.apache.spark.sql.SQLContext(sc)

Spark Session (Spark 2.x)

- When you enter pyspark, a spark session is already created for you
- Use sc to refer to the spark context in the session
- #read in a text file

  text=sc.textFile("test.txt")

Ref. https://databricks.com/blog/2016/08/15/how-to-use-sparksession-in-apache-spark-2-0.html

map vs flatMap

- **Spark map** function expresses a one-to-one transformation. It transforms each element of a collection into one element of the resulting collection.
- While **Spark flatMap** function expresses a one-to-many transformation. It transforms each element to 0 or more elements.

```
# a function to split the text
def tokenlize(text):
    return text.split()

#flatMap applies to each word/token
words=text.flatMap(tokenlize)
words.take(3)

#map applies to each row or line of words/tokens
words2=text.map(tokenlize)
words2.take(3)
```

Lambda and Map function

- Python supports anonymous **functions** (i.e. **functions** defined without a name), using a construct called "**lambda**".
- **Lambda functions** come from functional programming languages and the **Lambda** Calculus. Since they are so small they may be written on a single line.
- #Use map to create (key, val) pairs

  wc=words.map(lambda x: (x,1))

- Watch (10 minutes): Python: Lambda, Map, Filter, Reduce Function https://youtu.be/cKlnR-CB3tk (Note that filter is a new function in Spark 3, see https://mungingdata.com/spark-3/array-exists-forall-transform-aggregate-zip_with/)

UDF (User-defined function in pyspark)

Register a function as a UDF

Python

```
def squared(s):
  return s * s
spark.udf.register("squaredWithPython", squared)
```

```
def tokenlize(text):
    return text.split()
```

Use UDF with DataFrames

Python

```
from pyspark.sql.functions import udf
from pyspark.sql.types import LongType
squared_udf = udf(squared, LongType())
df = spark.table("test")
display(df.select("id", squared_udf("id").alias("id_squared")))
```

https://docs.databricks.com/spark/latest/spark-sql/udf-python.html

Your Turn

- The above codes can be found in wordcount3.ipython
- We use the IMSE server to run the demonstration
- You can also use the Databricks community edition as well
- You can import the wordcount3.ipython notebook (in the Canvas>Files>Python>wordcount folder).
- Experiment with various bits of pyspark codes in the spark-sklearn-app-MSE.py in Databricks notebook.
- How do you save the wordcount results using Databrick?

Ref

- https://github.com/bbengfort/hadoop-fundamentals/tree/master/spark
- https://pythonexamples.org/pyspark-word-count-example/#:~:text=What%20have%20we%20done%20in%20PySpark%20Word%20Count%3F,-We%20created%20a&text=words%20is%20of%20type%20PythonRDD,using%20single%20space%20as%20separator.